Enunciado

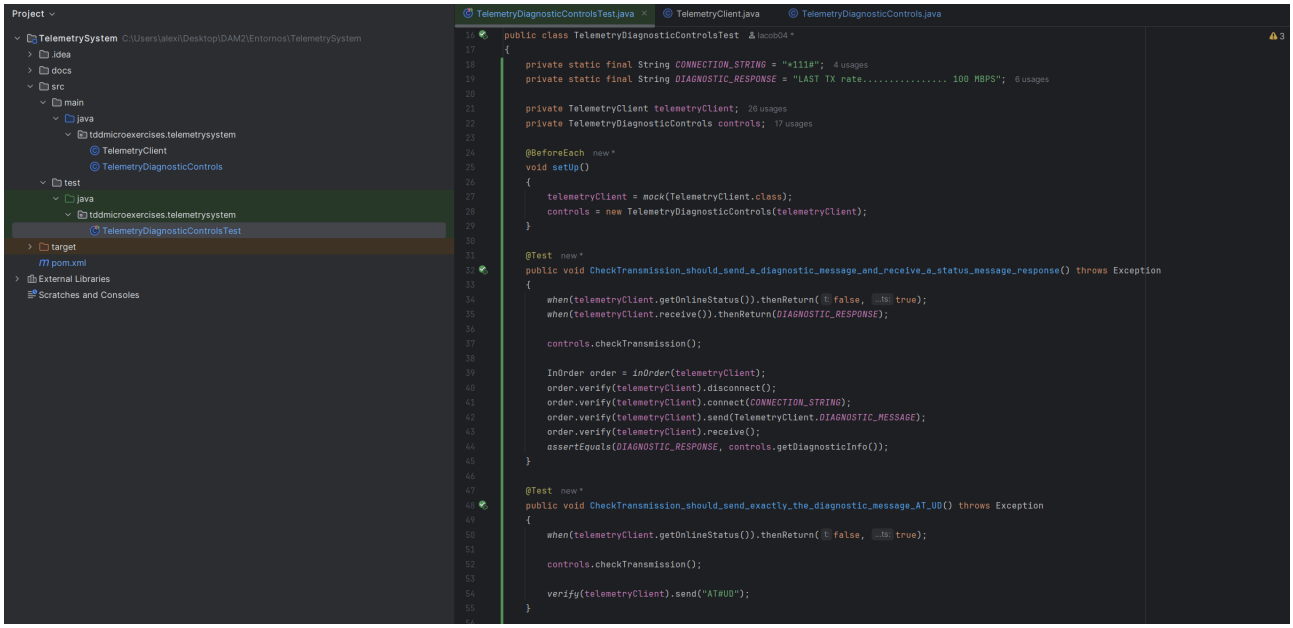
La situación

Acabas de empezar tu nuevo trabajo como desarrollador en el equipo Ferrari de Fórmula 1. Pero el código heredado no te convence del todo. Ahora quieres escribir pruebas unitarias, y eso está resultando más complicado de lo necesario. El fragmento de código no cumple con uno o más de los principios SOLID.

Aplica el estilo y el marco de pruebas unitarias con el que te sientas más cómodo. Debes usar mocks.

Tu tarea

Escribe las pruebas unitarias para la clase TelemetryDiagnosticControls. La clase TelemetryDiagnosticControls se encarga de establecer una conexión con el servidor de telemetría (a través de TelemetryClient), enviar una solicitud de diagnóstico y recibir correctamente la respuesta con la información de diagnóstico. La clase TelemetryClient proporcionada para el ejercicio simula el comportamiento de la clase real y puede responder con la información de diagnóstico o con una secuencia aleatoria. La clase real se conectaría y comunicaría con el servidor de telemetría mediante TCP/IP.


Project
TelemetrySystem C:\Users\alexi\Desktop\DAM2\Entornos\TelemetrySystem
.idea
docs
src
main
java
tddmicroexercises.telemetrysystem
TelemetryClient
TelemetryDiagnosticControls
test
java
tddmicroexercises.telemetrysystem
TelemetryDiagnosticControlsTest
target
pom.xml
External Libraries
Scratches and Consoles
TelemetryDiagnosticControlsTest.java
TelemetryClient.java
TelemetryDiagnosticControls.java
public class TelemetryDiagnosticControlsTest { 4 usages
private static final String CONNECTION_STRING = "*111#"; 4 usages
private static final String DIAGNOSTIC_RESPONSE = "LAST TX rate..... 100 MBPS"; 6 usages
private TelemetryClient telemetryClient; 26 usages
private TelemetryDiagnosticControls controls; 17 usages
@BeforeEach new *
void setUp()
telemetryClient = mock(TelemetryClient.class);
controls = new TelemetryDiagnosticControls(telemetryClient);
@Test new *
public void CheckTransmission_should_send_a_diagnostic_message_and_receive_a_status_message_response() throws Exception
when(telemetryClient.getOnlineStatus()).thenReturn(t: false, ...ts: true);
when(telemetryClient.receive()).thenReturn(DIAGNOSTIC_RESPONSE);
controls.checkTransmission();
InOrder order = inOrder(telemetryClient);
order.verify(telemetryClient).disconnect();
order.verify(telemetryClient).connect(CONNECTION_STRING);
order.verify(telemetryClient).send(TelemetryClient.DIAGNOSTIC_MESSAGE);
order.verify(telemetryClient).receive();
assertEquals(DIAGNOSTIC_RESPONSE, controls.getDiagnosticInfo());
@Test new *
public void CheckTransmission_should_send_exactly_the_diagnostic_message_AT_UD() throws Exception
when(telemetryClient.getOnlineStatus()).thenReturn(t: false, ...ts: true);
controls.checkTransmission();
verify(telemetryClient).send("AT#UD");

```java

    @Test  new *
    public void CheckTransmission_should_retry_connection_until_online() throws Exception
    {
        when(telemetryClient.getOnlineStatus()).thenReturn( t: false,  ...ts: false, false, true);
        when(telemetryClient.receive()).thenReturn(DIAGNOSTIC_RESPONSE);

        controls.checkTransmission();

        verify(telemetryClient, times( wantedNumberOfInvocations: 3)).connect(CONNECTION_STRING);
        verify(telemetryClient).send(TelemetryClient.DIAGNOSTIC_MESSAGE);
        assertEquals(DIAGNOSTIC_RESPONSE, controls.getDiagnosticInfo());
    }

    @Test  new *
    public void CheckTransmission_should_throw_after_three_failed_connection_attempts() throws Exception
    {
        when(telemetryClient.getOnlineStatus()).thenReturn( t: false);

        Exception ex = assertThrows(Exception.class, () -> controls.checkTransmission());

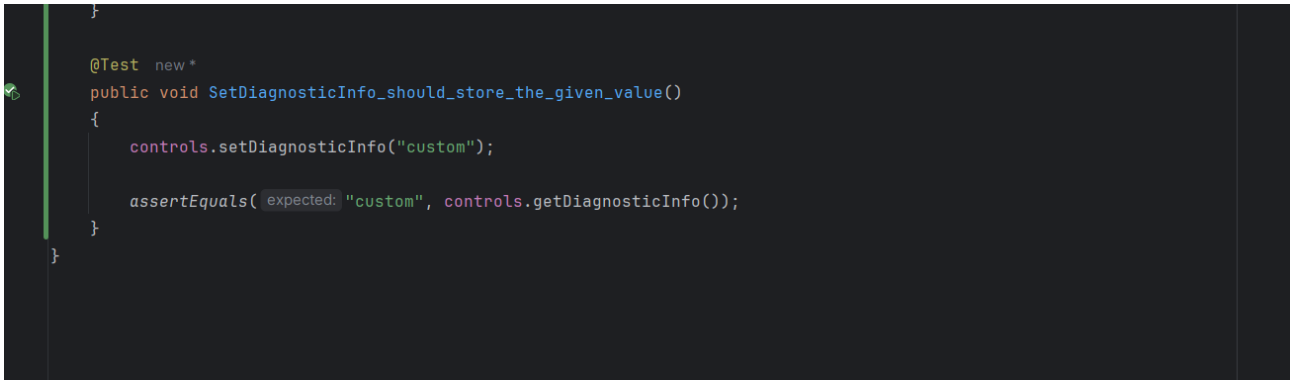
        assertEquals( expected: "Unable to connect.", ex.getMessage());
        verify(telemetryClient, times( wantedNumberOfInvocations: 3)).connect(CONNECTION_STRING);
    }

    @Test  new *
    public void CheckTransmission_should_not_send_nor_receive_when_connection_fails() throws Exception
    {
        when(telemetryClient.getOnlineStatus()).thenReturn( t: false);

        assertThrows(Exception.class, () -> controls.checkTransmission());

        verify(telemetryClient, never()).send(org.mockito.ArgumentMatchers.anyString());
        verify(telemetryClient, never()).receive();
    }
```

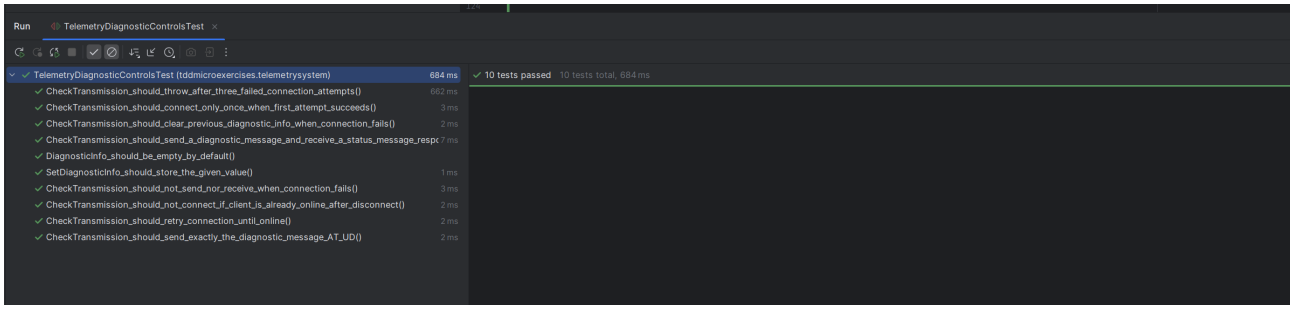
```
92      @Test  new *
93      public void CheckTransmission_should_clear_previous_diagnostic_info_when_connection_fails() throws Exception
94      {
95          controls.setDiagnosticInfo("old info");
96          when(telemetryClient.getOnlineStatus()).thenReturn( t: false);
97
98          assertThrows(Exception.class, () -> controls.checkTransmission());
99
100         assertEquals( expected: "", controls.getDiagnosticInfo());
101     }
102
103     @Test  Iacob04 *
104     public void CheckTransmission_should_connect_only_once_when_first_attempt_succeeds() throws Exception
105     {
106         when(telemetryClient.getOnlineStatus()).thenReturn( t: false,  ...ts: true);
107
108         controls.checkTransmission();
109
110         verify(telemetryClient, times( wantedNumberOfInvocations: 1)).connect(CONNECTION_STRING);
111     }
112
113     @Test  new *
114     public void CheckTransmission_should_not_connect_if_client_is_already_online_after_disconnect() throws Exception
115     {
116         when(telemetryClient.getOnlineStatus()).thenReturn( t: true);
117         when(telemetryClient.receive()).thenReturn(DIAGNOSTIC_RESPONSE);
118
119         controls.checkTransmission();
120
121         verify(telemetryClient, never()).connect(org.mockito.ArgumentMatchers.anyString());
122         assertEquals(DIAGNOSTIC_RESPONSE, controls.getDiagnosticInfo());
123     }
124
125     @Test  new *
126     public void DiagnosticInfo_should_be_empty_by_default()
127     {
128         assertEquals( expected: "", controls.getDiagnosticInfo());
129     }
130
```



```
    }

    @Test  new *
    public void SetDiagnosticInfo_should_store_the_given_value()
    {
        controls.setDiagnosticInfo("custom");

        assertEquals( expected: "custom", controls.getDiagnosticInfo());
    }
}
```


Run
TelemetryDiagnosticControlsTest
TelemetryDiagnosticControlsTest (tddmicroexercises.telemetrysystem) 684 ms
CheckTransmission_should_throw_after_three_failed_connection_attempts() 662 ms
CheckTransmission_should_connect_only_once_when_first_attempt_succeeds() 3 ms
CheckTransmission_should_clear_previous_diagnostic_info_when_connection_fails() 2 ms
CheckTransmission_should_send_a_diagnostic_message_and_receive_a_status_message_respc 7 ms
DiagnosticInfo_should_be_empty_by_default()
SetDiagnosticInfo_should_store_the_given_value() 1 ms
CheckTransmission_should_not_send_nor_receive_when_connection_fails() 3 ms
CheckTransmission_should_not_connect_if_client_is_already_online_after_disconnect() 2 ms
CheckTransmission_should_retry_connection_until_online() 2 ms
CheckTransmission_should_send_exactly_the_diagnostic_message_AT_UD() 2 ms
10 tests passed 10 tests total, 684 ms